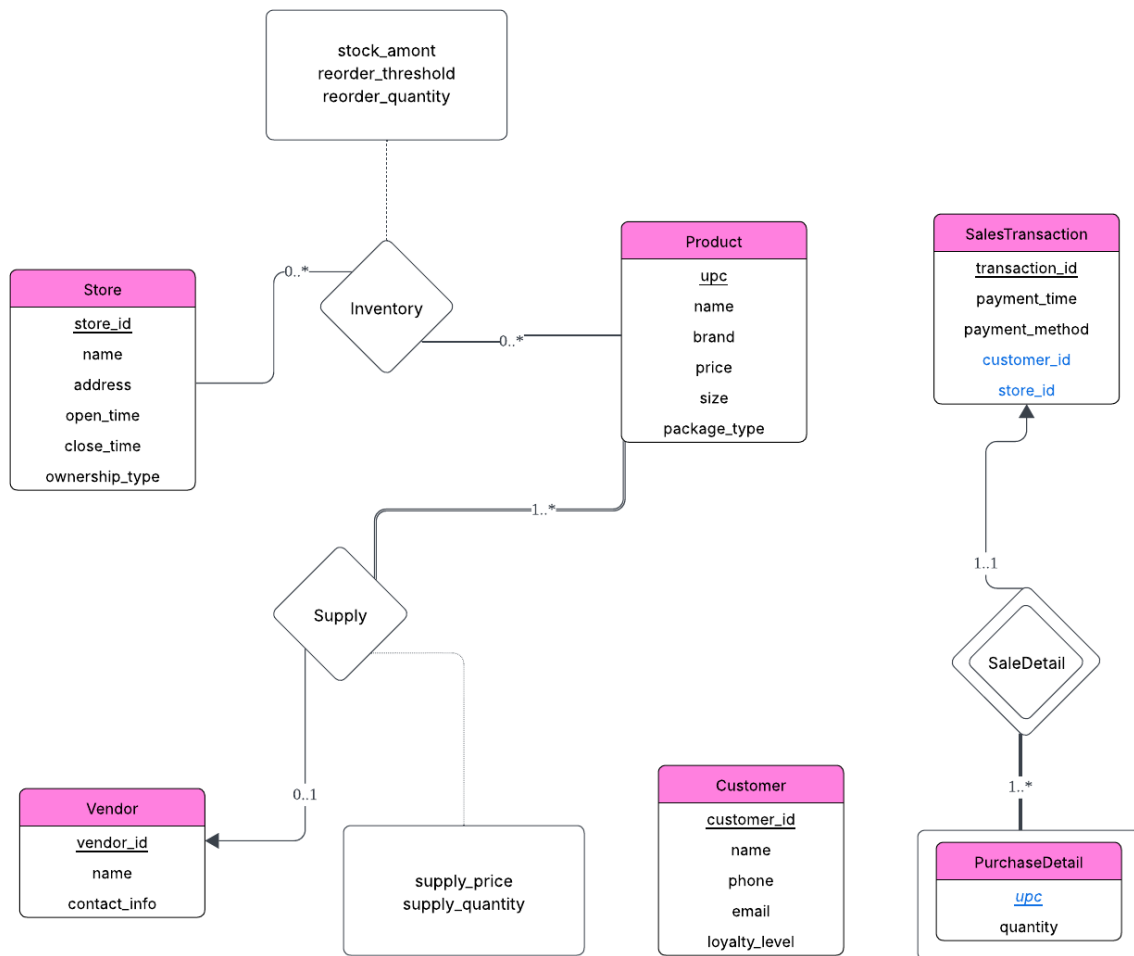


Database Systems

Spring 2025

Project 1

20231609 정희선



1. Short Description / Business Rules

위의 ER 모델은 다음 설계 가정과 비즈니스 규칙을 따릅니다.

- 각 Store는 고유한 ID를 가지며, 운영 형태는 직영과 가맹으로 구분됩니다. 등록된 지점은 제품을 보유하지 않을 수도 있고 (폐점 또는 DB에 등록만 되고 실제 운영 전 단계인 경우), 다양한 제품을 보유할 수 있습니다.
- 각 Product는 고유한 UPC를 가지며, **하나의 Vendor로부터 공급받는 것으로 가정합니다.**
- Customer는 DB에 등록되는 순간 기본 로열티 레벨을 가지며, 판매 이력은 SalesTransaction과 그에 따른 PurchaseDetail로 기록됩니다.

- 모든 거래는 반드시 하나 이상의 상품을 포함하며, 빈 거래는 허용되지 않습니다.
- 제품의 유통은 Supply(공급 관계)에 따라 이뤄집니다.
- Inventory(재고)는 Store와 Product의 관계로 표현되며, 재고 수량과 재주문 임계치를 포함합니다.

2. Entity and Relationship Justification

Entity는 사각형으로, relationship은 다이아몬드로 표현하였습니다. Weak entity는 이중 사각형으로, weak entity와 identifying strong entity를 연결하는 relationship은 이중 다이아몬드로 표현하였습니다.

각 entity의 **primary key**는 밑줄을 그어 표시하였고, **foreign key**는 파란색 글자로 표시하였습니다. Weak entity를 식별하기 위해 필요한 identifying entity의 속성은 weak entity 내부에 속성으로 작성하지 않았습니다. **Weak entity의 discriminator**는 밑줄 후 기울임체로 표현하였습니다. (강의자료 6.43에서는 discriminator를 점선 밑줄로 표현하나, lucid chart에서 해당 기능을 찾지 못하여 위 방식으로 대체하였습니다.)

각 entity와 relationship은 다음과 같이 정의되었습니다.

- **Store**: 지점을 독립된 실체로 정의하였고, 매장 고유 ID와 매장 이름, 주소, 운영시간 (open time, close time), 운영 형태(직영/가맹)를 저장하였습니다.
- **Product**: 제품은 식별 가능한 실체로 정의하였고, 고유 UPC와 제품 이름, 브랜드, 가격, 용량, 포장 유형이라는 속성들을 가집니다. (ex. 1-햇반 컵반 불고기-햇반-5000-210g-cup)
- **Vendor**: 공급 업체는 제품을 유통하는 실체로, 고유 ID와 이름, 연락처 정보의 속성을 가집니다.
- **Customer**: 고객은 고유한 customer id로 식별되며, 이름, 전화번호, email, loyalty 레벨이라는 속성을 가집니다.
- **SalesTransaction**: 각 거래를 실체로 만들어 고유한 거래 ID, 거래 시간, 거래 방법, 거래에 참여한 고객과 지점 id를 속성으로 가집니다.
- **PurchaseDetail**: 거래 안의 개별 제품 구매 정보를 담는 약한 entity로, SalesTransaction의 transaction_id와 제품 UPC로 식별됩니다. 구매한 제품 수를 속성으로 가집니다.
- **Inventory**: Store와 Product 사이의 관계로, 재고 수량, 재주문 임계치, 재주문량을 속성으로 가집니다.
- **Supply**: Vendor와 Product 사이의 관계로, 공급 단가와 공급량을 속성으로 가집니다.

- **SaleDetail:** SalesTransaction과 PurchaseDetail의 식별관계를 나타내는 관계입니다.

3. Mapping Cardinality Explanation

각 관계의 cardinality와 participation은 다음과 같이 설정하였습니다.

- **Inventory:** many-to-many 관계를 가지고, Store과 Product 모두 optional participation입니다. 즉, 하나의 Store는 여러 개의 Product를 가질 수 있으며, 하나의 Product 역시 여러 개의 Store에 존재할 수 있습니다. 또한, 폐업하거나 등록한지 얼마 안 된 Store는 Product를 하나도 가지지 않을 수 있고, 새로 나온 Product는 어떤 Store도 가지지 않을 수 있습니다.
- **Supply:** one(Vendor)-to-many(Product) 관계이며, Vendor는 optional participation이고 Product는 total participation입니다. 즉, 하나의 Vendor는 여러 개의 Product를 유통할 수 있으며, 하나의 Product는 여러 개의 Vendor에 의해 유통될 수 없습니다. 또한, 새로 등록된 Vendor는 어떤 Product도 유통하지 않을 수 있으나, Product는 무조건 특정 Vendor에 의해 유통되어야 합니다.
- **SaleDetail:** one(SalesTransaction)-to-many(PurchaseDetail) 관계이며, SalesTransaction과 PurchaseDetail 모두 total participation입니다. 즉, 하나의 SalesTransaction는 여러 개의 PurchaseDetail를 가질 수 있으며, 하나의 PurchaseDetail는 반드시 하나의 SalesTransaction과 연결됩니다. 또한, 모든 SalesTransaction은 PurchaseDetail을 가져야 하고, PurchaseDetail 역시 특정 SalesTransaction과 관련되어야 합니다.

4. Query Support Description

- **Product Availability:** Product.upc를 조건으로 Inventory를 통해 해당 제품을 보유한 Store와 재고 수량을 조회할 수 있습니다.
- **Top-Selling Items:** SalesTransaction.payment_time을 최근 한 달로 필터링한 후, SalesTransaction.store_id와 PurchaseDetail.upc별로 sum(PurchaseDetail.quantity)를 계산하여 매장 별 가장 많이 팔린 상품을 식별할 수 있습니다.
- **Store Performance:** PurchaseDetail.quantity * Product.price으로 항목 별 매출을 계산한 뒤, SalesTransaction.payment_time으로 분기를 설정합니다. 이후 SalesTransaction.store_id 별로 매출의 총합을 계산하여 가장 큰 값을 가진 지점을 출력할 수 있습니다.
- **Vendor Statistics:** Supply relation에서 Supply.vendor_id를 기준으로 Product.upc 개수를 집계하여 가장 다양한 제품을 공급하는 Vendor를 찾을 수 있습니다.
- **Inventory Reorder Alerts:** Inventory relation에서 where stock_amount < reorder_threshold 조건을 만족하는 Store와 Product 쌍을 구할 수 있습니다.

- **Customer Purchase Patterns:** Customer의 loyalty_level이 VIP인 경우를 필터링한 후, 해당 고객들의 SalesTransaction.transaction_id를 수집한 뒤 각 거래에서 PurchaseDetail을 조회합니다. Product.name으로 거래에서 커피의 유무를 판별한 뒤, 이 거래에서 등장한 다른 상품들을 count하여 상위 3개를 추출할 수 있습니다.
- **Franchise vs. Corporate Comparison:** Inventory relation을 기준으로 store_id별 distinct product count를 집계합니다. 이후 Store의 ownership_type 속성을 기준으로 구분하고 각 그룹 내에서 가장 많은 상품을 가진 Store를 비교할 수 있습니다.